

TASK 1 – TABLE WITHOUT INDEX

EXPLAIN command

EXPLAIN
SELECT * FROM labs.test_index_plan
WHERE load_date BETWEEN '2021-09-01 0:00' AND '2021-10-31 11:59:59'
ORDER BY 1;

2	Results 1
IN SELECT * FROM labs.test_index_plan WHERE Enter a SQL expression to filter results (use Ctrl+Space)	
AZ QUERY PLAN	
Sort (cost=328348.21..328545.42 rows=78884 width=16)	
Sort Key: num	
-> Seq Scan on test_index_plan (cost=0.00..321932.00 rows=78884 width=16)	
Filter: ((load_date >= '2021-09-01 00:00:00+03':timestamp with time zone) AND (load_date <= '2021-10-31 11:59:59+02':timestamp with time zone))	

EXPLAIN (ANALYZE, BUFFERS) (by default with buffers due to postgres preferences)

EXPLAIN ANALYZE
SELECT * FROM labs.test_index_plan
WHERE load_date BETWEEN '2021-09-01 0:00' AND '2021-10-31 11:59:59'
ORDER BY 1;

Results 2	Results 1	Results 3
PLAIN ANALYZE SELECT * FROM labs.test_index_pla Enter a SQL expression to filter results (use Ctrl+Space)		
AZ QUERY PLAN		
Sort (cost=382563.51..383913.19 rows=539873 width=16) (actual time=1201.298..1264.804 rows=523080.00 loops=1)		
Sort Key: num		
Sort Method: external merge Disk: 13320kB		
Buffers: shared hit=15961 read=69322, temp read=1665 written=1672		
-> Seq Scan on test_index_plan (cost=0.00..321932.00 rows=539873 width=16) (actual time=930.827..1024.537 rows=523080.00 loops=1)		
Filter: ((load_date >= '2021-09-01 00:00:00+03':timestamp with time zone) AND (load_date <= '2021-10-31 11:59:59+02':timestamp with time zone))		
Rows Removed by Filter: 15253560		
Buffers: shared hit=15958 read=69322		
Planning:		
0	Buffers: shared hit=19 dirtied=1	
1	Planning Time: 1.036 ms	
2	Execution Time: 1295.762 ms	

Conclusion:

Plain EXPLAIN command without execution predicted the significantly lower time/rows(78884) count that was denied by following real run - ANALYZE part of the query (rows count 523080). BUFFERS part adds one more layer of details — showing that even repeated runs still hit disk heavily (shared hit vs read), because the table's size exceeds Postgres's shared buffer cache.

TASK 2 – ADDING INDEX

B-tree Index added

1st run

Sort (cost=80274.79..81624.46 rows=539868 width=16) (actual time=305.886..524.334 rows=523080.00 loops=1)
Sort Key: num
Sort Method: external merge Disk: 13320kB
Buffers: shared hit=4260, temp read=1665 written=1672
-> Index Scan using idx_test_index_plan_btree on test_index_plan (cost=0.43..19643.80 rows=539868 width=16) (actual time=0.025..84.437 rows=523080.00 loops=1)
Index Cond: ((load_date >= '2021-09-01 00:00:00+03':timestamp with time zone) AND (load_date <= '2021-10-31 11:59:59+02':timestamp with time zone))
Index Searches: 1
Buffers: shared hit=4260
Planning Time: 0.086 ms
Execution Time: 618.000 ms

2nd run

AZ QUERY PLAN
Sort (cost=80274.79..81624.46 rows=539868 width=16) (actual time=260.888..326.975 rows=523080.00 loops=1)
Sort Key: num
Sort Method: external merge Disk: 13320kB
Buffers: shared hit=4260, temp read=1665 written=1672
-> Index Scan using idx_test_index_plan_btree on test_index_plan (cost=0.43..19643.80 rows=539868 width=16) (actual time=0.024..75.250 rows=523080.00 loops=1)
Index Cond: ((load_date >= '2021-09-01 00:00:00+03':timestamp with time zone) AND (load_date <= '2021-10-31 11:59:59+02':timestamp with time zone))
Index Searches: 1
Buffers: shared hit=4260
Planning Time: 0.112 ms
Execution Time: 357.503 ms

Conclusion: The index scan improves query time drastically. Running the same B-Tree indexed query twice showed that execution time improved on the second run (618 ms to 357.5 ms), this improvement likely reflects caching of Postgres.

Index Only Scan runs the entire query using only data stored in the index itself, without touching the actual table at all

AZ QUERY PLAN
Index Only Scan using idx_test_index_plan_btree on test_index_plan (cost=0.43..16721.80 rows=539868 width=8) (actual time=0.197..62.033 rows=523080.00 loops=1)
Index Cond: ((load_date >= '2021-09-01 00:00:00+03':timestamp with time zone) AND (load_date <= '2021-10-31 11:59:59+02':timestamp with time zone))
Heap Fetches: 0
Index Searches: 1
Buffers: shared hit=1433
Planning Time: 0.113 ms
Execution Time: 85.036 ms

Adding BRIN index

Sort (cost=294921.56..296271.23 rows=539868 width=16) (actual time=351.413..454.108 rows=523080.00 loops=1)
Sort Key: num
Sort Method: external merge Disk: 13320kB
Buffers: shared hit=2948, temp read=1665 written=1672
-> Bitmap Heap Scan on test_index_plan (cost=155.70..234290.56 rows=539868 width=16) (actual time=2.539..94.523 rows=523080.00 loops=1)
Recheck Cond: ((load_date >= '2021-09-01 00:00:00+03':timestamp with time zone) AND (load_date <= '2021-10-31 11:59:59+02':timestamp with time zone))
Rows Removed by Index Recheck: 21560
Heap Blocks: lossy=2944
Buffers: shared hit=2948
-> Bitmap Index Scan on idx_test_index_plan_brin (cost=0.00..20.74 rows=544022 width=0) (actual time=0.252..0.253 rows=29440.00 loops=1)
Index Cond: ((load_date >= '2021-09-01 00:00:00+03':timestamp with time zone) AND (load_date <= '2021-10-31 11:59:59+02':timestamp with time zone))
Index Searches: 1
Buffers: shared hit=4
Planning:
Buffers: shared hit=1
Planning Time: 0.107 ms
Execution Time: 514.715 ms

Conclusion: The Bitmap Index total execution time (495-515 ms) was faster than the no-index baseline but slightly slower than the equivalent B-Tree query (357–618 ms), since BRIN's approximation costs some extra heap recheck work that a precise B-Tree lookup avoids (Bitmap Heap Scan and Bitmap Index Scan)

TASK 3 BULK INSERT

Inserting data to table with index

```
--task3
CREATE TABLE labs.test_inserts (
    num          float NOT NULL,
    load_date    timestampz NOT NULL
);

CREATE INDEX idx_test_inserts_btree ON labs.test_inserts USING btree (load_date);

EXPLAIN ANALYZE
INSERT INTO labs.test_inserts (num, load_date)
SELECT num, load_date FROM labs.test_index_plan;
```

2

Results 1

Results 3

Results 5

Statistics 5

Results 6

Results 7

×

N ANALYZE INSERT INTO labs.test_inserts (num, load_date) Enter a SQL expression to filter results (use Ctrl+Space)

AZ QUERY PLAN
Insert on test_inserts (cost=0.00..243046.40 rows=0 width=0) (actual time=37028.434..37028.435 rows=0.00 loops=1)
Buffers: shared hit=32129538 read=85281 dirtied=128542 written=225129
-> Seq Scan on test_index_plan (cost=0.00..243046.40 rows=15776640 width=16) (actual time=0.457..993.874 rows=15776640.00 loops=1)
Buffers: shared read=85280 written=14
Planning:
Buffers: shared hit=3
Planning Time: 0.328 ms
Execution Time: 37028.924 ms

Every row inserted also requires inserting a corresponding entry into the index structure. This insert takes longer than an equivalent insert into an unindexed table.

Insert queries are rewritten to single query to avoid running 14 separate statements that forces PostgreSQL to open a distinct transaction, parse the syntax, update constraints/indexes, and write a separate log entry to disk 14 individual times.

```
INSERT INTO labs.emp (empno, ename, job, mgr, hiredate) VALUES
(1, 'SMITH', 'CLERK', 13, '1980-12-17'),
(2, 'ALLEN', 'SALESMAN', 6, '1981-02-20'),
(3, 'WARD', 'SALESMAN', 6, '1981-02-22'),
(4, 'JONES', 'MANAGER', 9, '1981-04-02'),
(5, 'MARTIN', 'SALESMAN', 6, '1981-09-28'),
(6, 'BLAKE', 'MANAGER', 9, '1981-01-05'),
(7, 'CLARK', 'MANAGER', 9, '1981-06-09'),
(8, 'SCOTT', 'ANALYST', 4, '1987-04-19'),
(9, 'KING', 'PRESIDENT', NULL, '1981-11-17'),
(10, 'TURNER', 'SALESMAN', 6, '1981-09-08'),
(11, 'ADAMS', 'CLERK', 8, '1987-05-23'),
(12, 'JAMES', 'CLERK', 6, '1981-12-03'),
(13, 'FORD', 'ANALYST', 4, '1981-12-03'),
(14, 'MILLER', 'CLERK', 7, '1982-01-23');
```

TASK 4 COPY COMMAND

```
--task4
COPY labs.test_index_plan TO 'C:/datasets/test_index_plan.csv'
CSV HEADER DELIMITER ',' FORCE QUOTE load_date;

COPY (
  SELECT * FROM labs.test_index_plan
  WHERE load_date BETWEEN '2021-09-01 0:00' AND '2021-09-01 11:59:59'
) TO 'C:/datasets/test_index_plan_short.csv'
CSV HEADER DELIMITER ',';

CREATE TABLE labs.test_copy (
  num float NOT NULL,
  load_date timestampz NOT NULL );

CREATE INDEX idx_test_copy_load_date ON labs.test_copy(load_date);

COPY labs.test_copy FROM 'C:/datasets/test_index_plan.csv'
CSV HEADER DELIMITER ',';
```

Name	Value
Updated Rows	15776640
Execute time	6.54s
Start time	Sat Jul 11 12:26:51 EEST 2026
Finish time	Sat Jul 11 12:26:58 EEST 2026
Query	COPY labs.test_index_plan TO 'C:/datasets/test_index_plan.csv'

Using COPY command, records were exported from labs.test_index_plan to a CSV file. Filtered export was executed to create a subset (test_index_plan_short.csv) for a specific date. The full

dataset was then sent back into a newly created and B-Tree indexed table (labs.test_copy)

TASK 5 UPSERT

The UPSERT command updates existing records:

Example:

emp 1 × emp 4 emp 3						
select * from labs.emp Enter a SQL expression to filter results (use Ctrl+Space)						
Grid	123 empno	A-Z ename	A-Z job	123 mgr	hiredate	
1	1	SMITH	CLERK	13	1980-12-17	
2	2	ALLEN	SALESMAN	6	1981-02-20	
3	3	WARD	SALESMAN	6	1981-02-22	
4	4	JONES	MANAGER	9	1981-04-02	
5	5	MARTIN	SALESMAN	6	1981-09-28	
6	6	BLAKE	MANAGER	9	1981-01-05	
7	7	CLARK	MANAGER	9	1981-06-09	

select * from labs.emp order by empno Enter a SQL expression to filter results (use Ctrl+Space)						
Grid	123 empno	A-Z ename	A-Z job	123 mgr	hiredate	
1	1	SMITH	MANAGER	13	2001-12-21	
2	2	ALLEN	SALESMAN	6	1981-02-20	
3	3	WARD	SALESMAN	6	1981-02-22	
4	4	JONES	ANALIST	9	2001-12-21	
5	5	MARTIN	SALESMAN	6	1981-09-28	
6	6	BLAKE	MANAGER	9	1981-01-05	
7	7	CLARK	MANAGER	9	1981-06-09	